

Structures and STRUC

■ Defining Data Structures:

```
struc Person  
    .id:  resd 1  
    .name: resb 32  
    .age:  resd 1  
endstruc
```

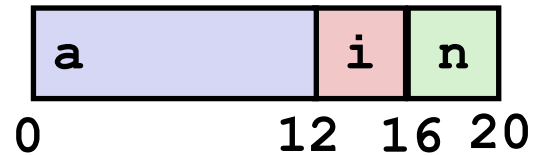
■ Accessing: `mov eax, [rbx + Person.age]`

■ NASM structures are essentially just offsets. They provide a clean way to manage complex data memory offsets without manual math.

Structure Allocation

```
struct rec {  
    int a[3];  
    int i;  
    struct rec *n;  
};
```

Memory Layout

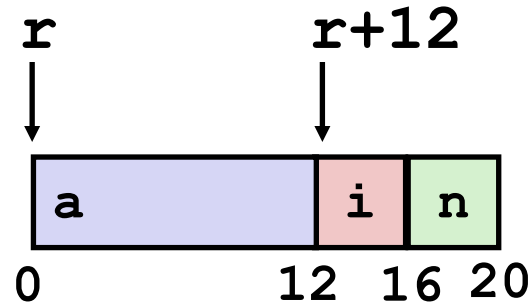


■ Concept

- Contiguously-allocated region of memory
- Refer to members within structure by names
- Members may be of different types

Structure Access

```
struct rec {  
    int a[3];  
    int i;  
    struct rec *n;  
};
```



■ Accessing Structure Member

- Pointer indicates first byte of structure
- Access elements with offsets

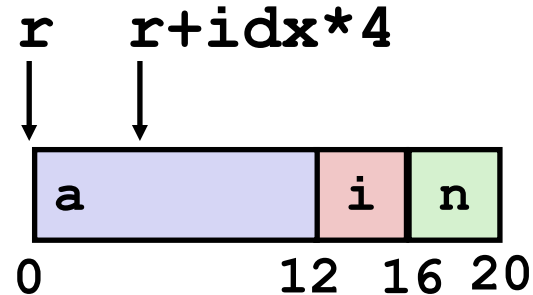
```
void  
set_i(struct rec *r,  
      int val)  
{  
    r->i = val;  
}
```

IA32 Assembly

```
# %edx = val  
# %eax = r  
movl %edx, 12(%eax) # Mem[r+12] = val
```

Generating Pointer to Structure Member

```
struct rec {  
    int a[3];  
    int i;  
    struct rec *n;  
};
```



■ Generating Pointer to Array Element

- Offset of each structure member determined at compile time
- Arguments
 - `Mem[%ebp+8]: r`
 - `Mem[%ebp+12]: idx`

```
int *get_ap  
(struct rec *r, int idx)  
{  
    return &r->a[idx];  
}
```

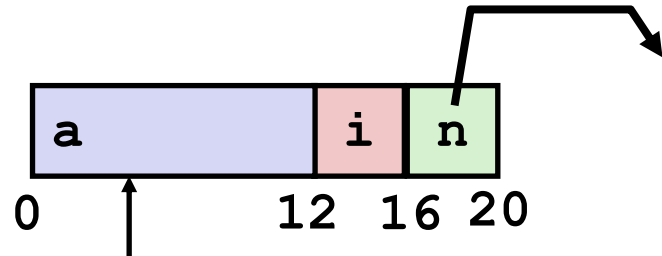
```
movl    12(%ebp), %eax    # Get idx  
sall    $2, %eax         # idx*4  
addl    8(%ebp), %eax     # r+idx*4
```

Following Linked List

■ C Code

```
void set_val
(struct rec *r, int val)
{
    while (r) {
        int i = r->i;
        r->a[i] = val;
        r = r->n;
    }
}
```

```
struct rec {
    int a[3];
    int i;
    struct rec *n;
};
```



Element i

Register	Value
%edx	r
%ecx	val

```
.L17:                                # loop:
    movl    12(%edx), %eax            # r->i
    movl    %ecx, (%edx,%eax,4)      # r->a[i] = val
    movl    16(%edx), %edx           # r = r->n
    testl   %edx, %edx               # Test r
    jne     .L17                     # If != 0 goto loop
```

Summary

■ Procedures in x86-64

- Stack frame is relative to stack pointer
- Parameters passed in registers

■ Arrays

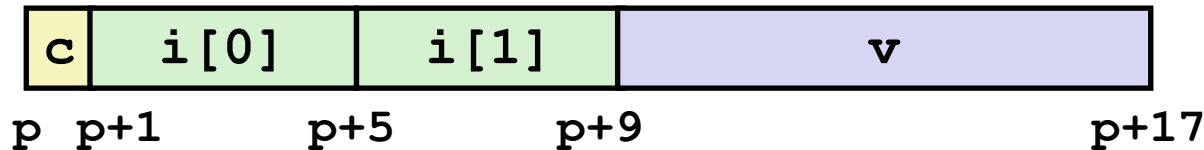
- One-dimensional
- Multi-dimensional (nested)
- Multi-level

■ Structures

- Allocation
- Access

Structures & Alignment

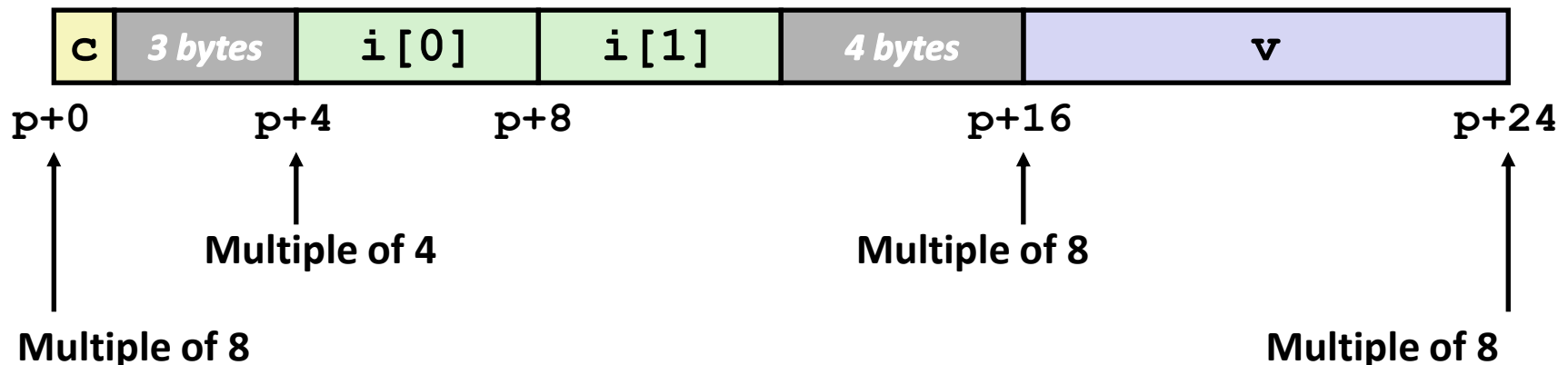
■ Unaligned Data



```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```

■ Aligned Data

- Primitive data type requires K bytes
- Address must be multiple of K



Alignment Principles

■ Aligned Data

- Primitive data type requires K bytes
- Address must be multiple of K
- Required on some machines; advised on IA32
 - treated differently by IA32 Linux, x86-64 Linux, and Windows!

■ Motivation for Aligning Data

- Memory accessed by (aligned) chunks of 4 or 8 bytes (system dependent)
 - Inefficient to load or store datum that spans quad word boundaries
 - Virtual memory very tricky when datum spans 2 pages

■ Compiler

- Inserts gaps in structure to ensure correct alignment of fields

Specific Cases of Alignment (IA32)

- **1 byte: char, ...**
 - no restrictions on address
- **2 bytes: short, ...**
 - lowest 1 bit of address must be 0_2
- **4 bytes: int, float, char *, ...**
 - lowest 2 bits of address must be 00_2
- **8 bytes: double, ...**
 - Windows (and most other OS's & instruction sets):
 - lowest 3 bits of address must be 000_2
 - Linux:
 - lowest 2 bits of address must be 00_2
 - i.e., treated the same as a 4-byte primitive data type
- **12 bytes: long double**
 - Windows, Linux:
 - lowest 2 bits of address must be 00_2
 - i.e., treated the same as a 4-byte primitive data type

Specific Cases of Alignment (x86-64)

- **1 byte: char, ...**
 - no restrictions on address
- **2 bytes: short, ...**
 - lowest 1 bit of address must be 0_2
- **4 bytes: int, float, ...**
 - lowest 2 bits of address must be 00_2
- **8 bytes: double, char *, ...**
 - Windows & Linux:
 - lowest 3 bits of address must be 000_2
- **16 bytes: long double**
 - Linux, Windows:
 - lowest 3 bits of address must be 000_2
 - i.e., treated the same as a 8-byte primitive data type

Satisfying Alignment with Structures

■ Within structure:

- Must satisfy each element's alignment requirement

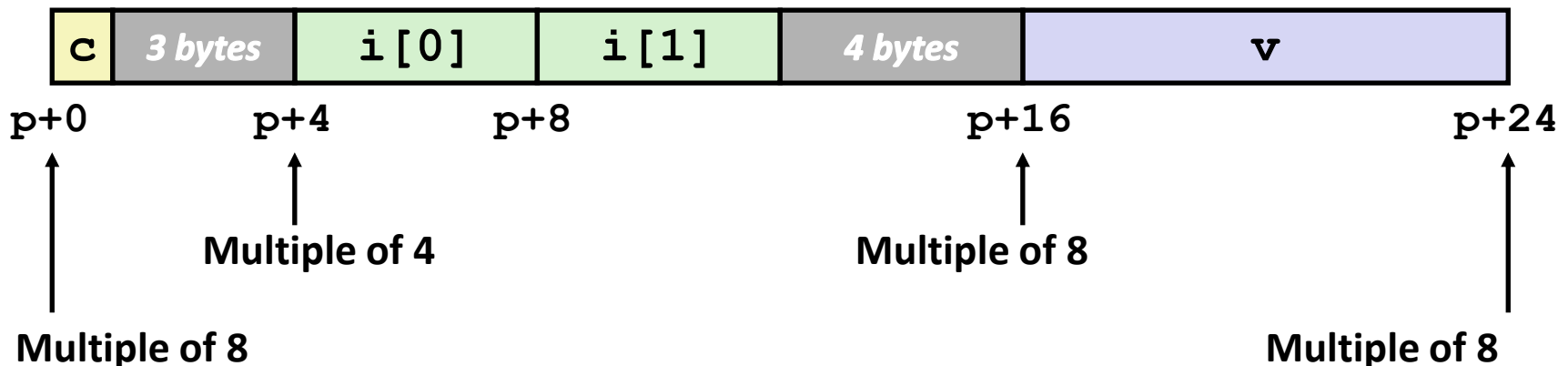
■ Overall structure placement

- Each structure has alignment requirement K
 - K = Largest alignment of any element
- Initial address & structure length must be multiples of K

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```

■ Example (under Windows or x86-64):

- $K = 8$, due to `double` element

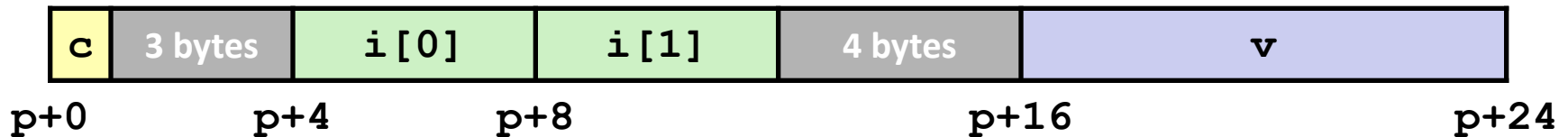


Different Alignment Conventions

■ x86-64 or IA32 Windows:

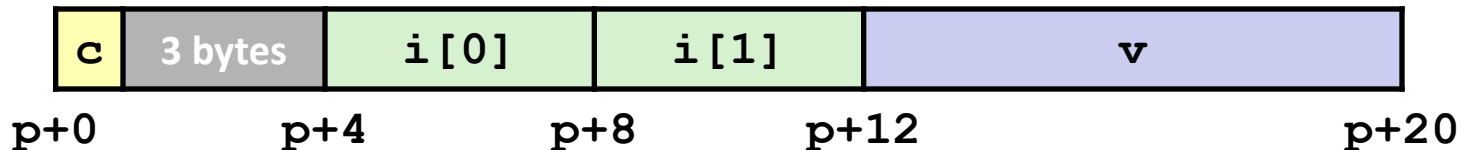
- $K = 8$, due to `double` element

```
struct S1 {  
    char c;  
    int i[2];  
    double v;  
} *p;
```



■ IA32 Linux

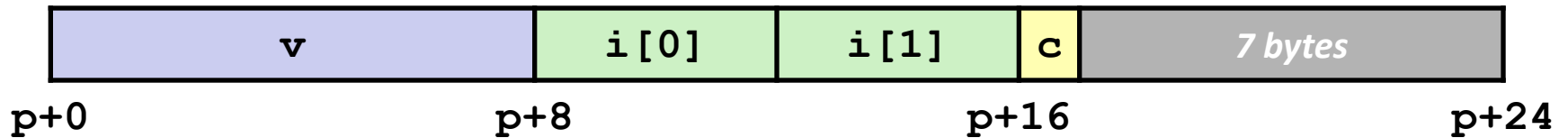
- $K = 4$; `double` treated like a 4-byte data type



Meeting Overall Alignment Requirement

- For largest alignment requirement K
- Overall structure must be multiple of K

```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} *p;
```



Arrays of Structures

- Overall structure length multiple of K
- Satisfy alignment requirement for every element

```
struct S2 {  
    double v;  
    int i[2];  
    char c;  
} a[10];
```

